SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: MLA0304
COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Dumpala Prasanth

QUESTIONS: 10 **Total Marks:50**

Annexure A

Constraint - Based Problem-Solving **Duration: 2Hrs**

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

   (10 Marks)
2. Implement an ε-greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

   (10 Marks)
3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

   (10 Marks)
4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

   (10 Marks)

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ε-greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

Code of Conduct:

I ***_K varshan_ 192325089*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

192325089.
K. YARSHINI

1) Grid world Navigation using OPEN AI Gym

Definition.

State Space : The agents position in the grid (x, y)

Action Space : up, down, left, right.

Reward function:

1. +10 for reaching the goal
2. -10 for hitting an obstacle.
3. -1 for each step Taken.

constraints

1.) Agent cannot move outside the grid.

2.) obstacles must be avoided.

3.) Episode ends when the goal is reached.

program:

```
Import gym.
form gym import spaces.
import numpy as np.
class grid world (gym.Env):
    def __init__(self):
        super().__init__()
        self.size = 5.
```

```
self.goal = (4,4)
self.obstacle = (2,2)
self.action_space =
    self.observation_space =
        spaces.Box(low=0, high=4, shape=(2,),
env = GridWorldEnv()
print("Initial state:", env.state)
```

2) ε-greedy multi armed bandit

```
import random
epsilon = 0.1
trials = 500
arms = [0.2, 0.5, 0.8]
counts = [0]*3
values = [0]*3
Total_reward = 0
for _ in range(trials):
    if random.random() < epsilon:
        arm = random.randint(0,2)
    else:
        arm = values.index(max(values))
        reward = 1 if
```

Value Iteration using python

program.

```
import numpy as np
gramma = 0.9
grid = np.(4,4).
goal = (3.3)
restricted = (1,1)
for _ in range (100).
new_grid = grid copy ().
    for i in range (4):
    for j in range (4):
        if (i, j) = goal or
i, j = = restricted = continue.
        values = ()
for dx, dy in ((1,0) (-1.0) (0.1), (0-1)):
    x = min (3, max (0. )+dx))
    y = min 3. max (0. j dx)).
        reward = -1
```

Prestricted states.

1.) cannot be visited.

2.) Value ignores such states while computing.

random.random() < arms[arm] else

counts[arm] += 1

values[arm] += (rewards - values[arm])
/ count[arm]

Total_reward += reward

print("Total reward:", total_reward)

E value	Exploration	Exploitation	constraint satisfaction
0.1	low	high	Better reward maximization
0.3	medium	Balanced	Moderate performance
0.5	high	low	More explor. but covers rewards

3) Markov Decision process for Autonomous Robot.

States:

1.) Robots position.

2.) Remaining battery level.

Actions

1. Move up.
2. Move down.
3. Move left.

4. move right.

5. Stay

Transition probability.

1) 90% correct movement.

2) 10% movement.

Rewards;

1) +100 for reaching destination.

2) -1 for every movement.

3) -10 for loose battery usage.

4) -50 if battery becomes zero before reaching.

5) Battery capacity is limited.

6) Robot cannot exceed available energy.

7) Shortest path is preferred.

MDP representation.

State → Action → Next state.

S1 → move right → S2

↓ ↓

move move right.
down

↓ ↓

S3 - - - - - - - goal.

5) RL framework for warehouse robot.

constraints:

1.) Battery limit = 30 minutes.

2.) Robot must maximize deliveries.

3.) charging consumes time.

State space.

1.) Robot location.

2.) Battery percentage.

Rewards

successfully delivery = +20

Reset = +10

Battery consumption = +5.

collision = -10,

RL framework :